

AI Analytics Agent

Architecture & System Design Document

Version 1.0 | May 2026

1. Purpose & Scope

This document describes the architecture, component design, data flow, and deployment strategy of the AI Analytics Agent — a Streamlit-based application that converts raw CSV/Excel data into an automated, insight-driven analytics dashboard powered by a large language model (LLM).

The agent is designed to serve business analysts, data enthusiasts, and domain experts who need rapid insights from tabular data without writing code or maintaining BI tooling.

2. System Overview

2.1 High-Level Architecture

The application follows a four-layer architecture:

Layer	Technology	Responsibility
Presentation	Streamlit	Web UI — sidebar controls, tabbed dashboard, chart rendering
Application	Python (src/ modules)	Orchestration of data ingestion, preprocessing, KPI detection, insight generation
AI / LLM	Groq API — Llama 3.3 70B	Metadata inference, KPI prioritisation, natural language insight and recommendation generation
Data	Pandas / openpyxl	File loading, cleaning, statistical summarisation, multi-sheet Excel support

2.2 Technology Stack

- Frontend & Hosting: Streamlit (Python web framework with reactive state)
- LLM Provider: Groq Cloud — Llama 3.3 70B Versatile (fast inference, free tier available)
- Data Processing: Pandas, NumPy, openpyxl
- Visualisation: Plotly Express & Graph Objects
- Deployment Target: Streamlit Community Cloud
- Secret Management: Streamlit Secrets (st.secrets) for API key injection

3. Module Breakdown

3.1 src/llm/groq_client.py

Wraps the Groq Python SDK into a thin client. Responsibilities:

- Validates the API key on instantiation
- Exposes a single chat() method that accepts a list of messages and returns a string
- Handles rate-limit retries and error propagation

3.2 src/data/ingestion.py

Handles file I/O for both CSV and multi-sheet Excel:

- `load_file()` detects file type from extension, returns {sheet_name: DataFrame} dict
- `get_data_summary()` produces a lightweight schema description (column names, dtypes, sample rows) for LLM consumption

3.3 src/data/preprocessing.py

Runs a deterministic cleaning pipeline on every sheet:

- Drops fully-duplicate rows
- Standardises column names (`lower_snake_case`)
- Parses date columns by heuristic detection
- Fills or flags missing values, generating a preprocessing report per sheet
- `get_statistical_summary()` computes `describe()` statistics for numeric and categorical columns

3.4 src/metadata/handler.py

Manages three metadata input pathways:

- User-supplied JSON file (`load_metadata_file` + `validate_metadata`)
- Manual form (`build_metadata_from_form`)
- LLM-inferred metadata (`infer_metadata_with_llm`) — invoked when column descriptions are absent

3.5 src/kpi/detector.py

The KPI detection engine operates in two phases:

- Phase 1 — Heuristic scan: iterates columns, matches names against known KPI patterns (revenue, churn, margin, etc.) and aggregates values
- Phase 2 — LLM ranking: sends the heuristic candidates plus metadata to the LLM, which assigns priority (high/medium/low), aggregation type, and a business description

3.6 src/insights/engine.py

Orchestrates insight generation via a structured LLM prompt:

- Positives — metrics performing well
- Concerns — anomalies, outliers, declining trends
- Trends — directional signals over time
- Recommendations — action items with rationale and expected impact
- Health Score (0–100) — a synthetic quality index returned as JSON

3.7 src/dashboard/charts.py

Stateless chart factory functions built on Plotly:

- `render_kpi_chart()` — bar, line, or pie based on KPI type
- `render_trend_chart()` — time-series line chart
- `render_distribution_charts()` — histograms and bar charts for each column
- `render_correlation_heatmap()` — Pearson correlation matrix
- `render_missing_values_chart()` — horizontal bar chart of null rates
- `render_kpi_priority_chart()` — bubble chart of KPIs by priority and value

4. Data Flow

Step	Action	Detail
1	File Upload	User uploads CSV or XLSX. <code>load_file()</code> returns a dict of DataFrames.
2	Metadata Input	User fills form, uploads JSON, or proceeds without — LLM infers column metadata.
3	Preprocessing	<code>preprocess_all()</code> cleans every sheet; a report dict records changes made.
4	Metadata Inference	If column metadata is absent, <code>infer_metadata_with_llm()</code> calls the LLM with a data summary.
5	KPI Detection	<code>detect_kpis()</code> runs heuristic + LLM pass, returns ranked KPI objects.
6	Insight Generation	<code>generate_insights()</code> sends KPIs + stats to LLM, receives structured JSON.
7	Dashboard Render	Streamlit renders charts, KPI cards, insight panels across five tabs.

5. State Management

Streamlit is stateless across reruns; all application data is stored in `st.session_state`:

Key	Contents
<code>sheets</code>	Raw <code>{sheet_name: DataFrame}</code> loaded from the uploaded file
<code>clean_sheets</code>	Preprocessed DataFrames after cleaning pipeline
<code>metadata</code>	Full metadata dict (user-supplied or LLM-inferred)
<code>kpis</code>	List of KPI objects with name, value, priority, chart config
<code>insights</code>	Insight dict: positives, concerns, trends, recommendations, <code>health_score</code>
<code>llm</code>	GroqClient instance (created on API key entry)
<code>processed</code>	Boolean flag — triggers dashboard render vs. landing page

6. LLM Integration Design

6.1 Prompt Strategy

All LLM calls follow a structured JSON-output pattern. The system prompt specifies the exact JSON schema expected; the model is instructed to return only valid JSON with no markdown fencing. Responses are parsed with a try/except that strips code fences if present.

6.2 Calls Made Per Analysis Run

- Metadata inference (optional): 1 call — given column names + sample rows, returns column descriptions
- KPI ranking: 1 call — given heuristic KPI list + metadata, returns priority/aggregation/description
- Insight generation: 1 call — given KPI values + statistical summary, returns full insight JSON

Total: 2–3 LLM calls per run, keeping latency and token usage minimal.

6.3 Groq Model Selection

The application targets Llama 3.3 70B Versatile on Groq. This model was chosen because:

- Sub-second inference latency on Groq infrastructure
- Strong instruction-following for structured JSON output
- Free tier with generous rate limits suitable for multi-user deployment

7. Deployment Architecture

7.1 Streamlit Community Cloud

The app is deployed on Streamlit Community Cloud (streamlit.io/cloud), which provides:

- GitHub-connected continuous deployment — push to main branch triggers redeploy
- Built-in HTTPS and custom subdomain
- Secrets management via the cloud dashboard (GROQ_API_KEY stored as st.secrets)

7.2 Secret Injection

The Groq API key is stored in Streamlit Cloud Secrets and accessed via `st.secrets['GROQ_API_KEY']`. The app initialises the `GroqClient` automatically on startup, removing the need for users to supply their own key.

7.3 Required Repository Files

- `app.py` — entry point
- `requirements.txt` — Python dependencies
- `src/` — application modules
- `.streamlit/secrets.toml` — local dev secret file (git-ignored)

7.4 requirements.txt

Core dependencies: streamlit, pandas, numpy, openpyxl, plotly, groq

8. Security Considerations

- API key is never exposed in client-side code; injected server-side via st.secrets
- Uploaded files exist only in Streamlit's in-memory session — not persisted to disk
- No user authentication required for a public demo; can be added via Streamlit Authenticator
- LLM prompts contain only statistical summaries, not raw row data, reducing PII exposure

9. Scalability & Limitations

- Streamlit Cloud free tier: single-process, ~1 GB RAM — suitable for files up to ~50 MB
- Concurrent users each get an isolated session; no shared state
- LLM calls are synchronous; high concurrency may hit Groq rate limits
- For production scale, replace Groq with a self-hosted or enterprise LLM endpoint

10. Future Enhancements

- Natural language query interface (text-to-pandas via LLM)
- Scheduled refresh via Streamlit + cron / GitHub Actions
- Export dashboard as PDF report
- Multi-user authentication and saved analyses
- Support for open-source LLMs via Ollama for fully on-premises deployment